Lab 4: Vibe Coding - Complete LLM Chat Transcript

Course / Module: Software Engineering Lab 4 (Vibe Coding)

Student SRN: PES1UG24AM106

Assigned Repo: 40_flappy-bird (Real-Time Flappy Bird Clone)

AI Pair-Programmer: Google DeepMind Gemini Assistant

Deliverable: Deliverable (c) - Complete LLM Chat History

Status: All 4 Tasks + Start Screen & Maximize Verified

USER (STUDENT) 2026-10-15 06:23:13

Real-Time Flappy Bird Clone

This project is a terminal-based Flappy Bird clone using Pygame. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

What's Provided

A partially working version of a Flappy Bird game with:

A player-controlled bird affected by gravity, with a flap on key/click

Scrolling pipes with a randomized gap

Score display

You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

Clone the repo or download the project folder.

Make sure you have Python 3.10+ installed.

Install dependencies:

pip install -r requirements.txt

Run the game:

python main.py

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

The bird sometimes clips the edge of a pipe without dying, especially as pipe speed increases. Investigate and enhance collision accuracy so hits are detected consistently.

Task 2: Implement Game Over Condition

Add a screen that displays the final score once the bird hits the ground, ceiling, or a pipe, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard pipe speed/gap), or exit.

Task 4: Add Sound Feedback

Add basic sound effects for flapping, passing a pipe (scoring), and dying.

Expected Behavior

Bird flaps upward on Space or mouse click, and falls due to gravity otherwise

Pipes spawn at a regular interval and scroll from right to left with a randomized gap

Score increases by one each time the bird passes a pipe

Game ends when the bird hits the ground, the ceiling, or a pipe

Folder Structure

flappybird-main/

```
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── bird.py
│   └── pipe.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

[] A 10-second video of gameplay before your changes, showing the bug/broken behavior

[] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working

[] The Chat/LLM used page link, with the complete chat history

THIS IS THE README.md FILE

game/bird.py

```
import pygame

class Bird:
def __init__(self, x, y, radius=15):
self.x = x
self.y = y
```

```
self.radius = radius
self.velocity = 0
self.gravity = 0.5
self.flap_strength = -8

def flap(self):
self.velocity = self.flap_strength

def update(self):
self.velocity += self.gravity
self.y += self.velocity

def center(self):
return (self.x, self.y)

def rect(self):
return pygame.Rect(self.x - self.radius, self.y - self.radius, self.radius *2, self.radius* 2)
```

game/game_engine.py

```
import pygame
from .bird import Bird
from .pipe import Pipe
```

Game Engine

```
WHITE = (255, 255, 255)

GREEN = (0, 150, 0)


class GameEngine:

def __init__(self, width, height):

self.width = width

self.height = height


self.bird = Bird(width // 4, height // 2)

self.pipe_speed = 4

self.pipe_interval = 90 # frames between pipe spawns

self._spawn_timer = 0

self.pipes = [Pipe(width + 100, height, speed=self.pipe_speed)]


self.score = 0

self.font = pygame.font.SysFont("Arial", 30)

self.game_over = False


def handle_event(self, event):

# Flap is edge-triggered (KEYDOWN / MOUSEBUTTONDOWN), not held.

if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:

self.bird.flap()
```

```
if event.type == pygame.MOUSEBUTTONDOWN:
self.bird.flap()

def handle_input(self):
# Reserved for continuously-held-key input; flapping is handled
# in handle_event instead, so there's nothing to poll here.
pass

def update(self):
if self.game_over:
return

self.bird.update()

if self.bird.y - self.bird.radius <= 0 or self.bird.y + self.bird.radius >= self.height:
self.game_over = True
return

self._spawn_timer += 1
if self._spawn_timer >= self.pipe_interval:
self._spawn_timer = 0
self.pipes.append(Pipe(self.width, self.height, speed=self.pipe_speed))

for pipe in self.pipes:
```

```
pipe.move()


# NOTE: collision only checks the bird's single center point
# against the pipe rects, rather than the bird's full rect.
# At higher pipe speeds the bird can visually clip a pipe's
# edge for a frame or two without this ever registering a
# hit. See Task 1 in the README.
if pipe.top_rect().collidepoint(self.bird.center()) or
pipe.bottom_rect().collidepoint(self.bird.center()):
self.game_over = True


if not pipe.scored and pipe.x + pipe.width < self.bird.x:
pipe.scored = True
self.score += 1


self.pipes = [p for p in self.pipes if not p.off_screen()]


def render(self, screen):
for pipe in self.pipes:
pygame.draw.rect(screen, GREEN, pipe.top_rect())
pygame.draw.rect(screen, GREEN, pipe.bottom_rect())


pygame.draw.circle(screen, WHITE, (int(self.bird.x), int(self.bird.y)), self.bird.radius)
```

```
score_text = self.font.render(f"Score: {self.score}", True, WHITE)
screen.blit(score_text, (10, 10))

if self.game_over and not getattr(self, "_game_over_logged", False):
# NOTE: no proper game-over screen yet - see Task 2 in the README.
print("Game over! Final score:", self.score)
self._game_over_logged = True
```

game/pipe.py

```
import pygame
import random

class Pipe:
def __init__(self, x, screen_height, gap=150, width=70, speed=4):
self.x = x
self.width = width
self.gap = gap
self.speed = speed
self.screen_height = screen_height
self.gap_y = random.randint(100, screen_height - 100 - gap)
self.scored = False

def move(self):
```

self.x -= self.speed

def off_screen(self):

return self.x + self.width < 0

def top_rect(self):

return pygame.Rect(self.x, 0, self.width, self.gap_y)

def bottom_rect(self):

bottom_y = self.gap_y + self.gap

return pygame.Rect(self.x, bottom_y, self.width, self.screen_height - bottom_y)

main.py

import pygame

from game.game_engine import GameEngine

Initialize pygame/Start application

pygame.init()

Screen dimensions

WIDTH, HEIGHT = 500, 700

SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))

pygame.display.set_caption("Flappy Bird - Pygame Version")

Colors

SKY_BLUE = (135, 206, 235)

WHITE = (255, 255, 255)

Clock

clock = pygame.time.Clock()

FPS = 60

Game loop

engine = GameEngine(WIDTH, HEIGHT)

def main():

running = True

while running:

SCREEN.fill(SKY_BLUE)

for event in pygame.event.get():

if event.type == pygame.QUIT:

running = False

engine.handle_event(event)

```
engine.handle_input()

engine.update()

engine.render(SCREEN)


pygame.display.flip()

clock.tick(FPS)


pygame.quit()


if __name__ == "__main__":

main()
```

```
requirements.txt

pygame
```

THIS IS WHAT EVERYTHING THAT I HAVE IN THE REPO

https://github.com/SETAPESU26/40_flappy-bird/tree/main

THIS IS THE URL OF THE REPO

I WANT YOU CREATE ME AN EXACT FOLDER STRUCTER IN THIS LOCATION

C:\SE_LAB4

AND TELL ME HOW EXACTLY TO RUN THIS TO RECORD THAT FIRST VIDEO

AFTER THAT YOU SUGGEST ME FEW SUGGESTIONS AND ILL ALSO SEE IF I HAVE ANY EXTRA

GIVE ME AN UPDATED CODE

WITH WHICH YOU CAN CREATE THE UPDATED VERSION OF THE GAME AND PUT ALL THOSE FILE IN THE STRUCTURE BUT IN A DIFFERENT SUB FOLDER IN THE SAME LOCATION

AI ASSISTANT (PAIR-PROGRAMMING PARTNER) 2026-10-15 06:30:42

Both the **original starter code** and the **fully completed, updated version** have been created in `C:\SE_LAB4`.

Folder Structure Created at `C:\SE_LAB4`

```
C:\SE_LAB4\
├── flappybird-main\              <-- [ORIGINAL / STARTER CODE with bugs]
│   ├── README.md
│   ├── requirements.txt
│   ├── main.py
│   └── game\
│       ├── __init__.py
│       ├── bird.py
│       ├── game_engine.py
│       └── pipe.py
│
```

```
└── flappybird-updated\            <-- [UPDATED / COMPLETED VERSION]

    ├── README.md

    ├── requirements.txt

    ├── main.py

    ├── assets\

    │   └── sounds\

    │       ├── flap.wav          <-- 16-bit retro jump sweep

    │       ├── score.wav         <-- Two-tone chime

    │       ├── hit.wav           <-- Impact collision thud

    │       └── generate_sounds.py

    └── game\

        ├── __init__.py

        ├── bird.py               <-- Task 1: Circle-to-AABB collision

        ├── pipe.py               <-- Proper gap & pipe rendering

        ├── sound_manager.py      <-- Task 4: Audio management
```

```
        └── game_engine.py          <-- Task 2 & 3: Game Over overlay & Replay/Difficul
```

---

1. How to Run & Record the FIRST Video (Broken Behavior)

Running the Original Game

Open PowerShell or Command Prompt and run:

```
cd C:\SE_LAB4\flappybird-main

python main.py
```

What to Show in Your 10-Second Video:

1. **The Edge-Clipping Collision Bug (Task 1)**:
2. Fly the bird so that its outer edge or top/bottom grazes the pipe edge.
3. **Why it happens**: In [game_engine.py](file:///C:/SE_LAB4/flappybird-main/game/game_engine.py#L58-L62), collision is checked with `collidepoint(self.bird.center())`. It only checks the 1-pixel center coordinate of the bird, ignoring its 15-pixel radius! The bird can visually penetrate 14 pixels into any pipe edge without dying.
4. **The Missing Game Over Screen (Task 2 & Task 3)**:
5. Let the bird crash into the ground or directly into the center of a pipe.
6. Notice the window completely **freezes** in place with no visual feedback. You cannot restart or press any key to try again; it only prints `Game over! Final score: X` to the terminal console.
7. **No Sound (Task 4)**:
8. Flapping, passing pipes, and dying are completely silent.

> **Screen Recording Tip**: Press <kbd>Win</kbd> + <kbd>Alt</kbd> + <kbd>R</kbd> (Windows Game Bar) or <kbd>Win</kbd> + <kbd>Shift</kbd> + <kbd>S</kbd> (Snipping Tool Screen Recorder) to record the 10-second clip.

---

2. Technical Suggestions & Code Review

Task 1: Accurate Circle-to-AABB Collision

- **Problem**: A point-check (`collidepoint(bird.center())`) misses the entire 15-pixel radius, while a simple box bounding box (`bird.rect()`) makes the corners trigger false hits.
- **Solution**: Exact **Circle-to-AABB** (Axis-Aligned Bounding Box) mathematical distance:

$$\text{closest}_x = \max(\text{rect.left}, \min(\text{bird.x}, \text{rect.right}))$$

$$\text{closest}_y = \max(\text{rect.top}, \min(\text{bird.y}, \text{rect.bottom}))$$

$$\Delta x = \text{bird.x} - \text{closest}_x, \quad \Delta y = \text{bird.y} - \text{closest}_y$$

$$\text{colliding} \iff \Delta x^2 + \Delta y^2 \le \text{radius}^2$$

Task 2: In-Game Game Over Screen

- Instead of freezing the window, overlay a semi-transparent card showing:
- "GAME OVER" title
- "Final Score" and persistent "Best Score"
- Visual menu for difficulties and replay prompts.

Task 3: Difficulty & Replay Logic

- Implement state resets without exiting the Python process:
- **Easy**: Speed: `3`, Gap: `185px`, Spawn interval: `105 frames`

- **Medium**: Speed: `4`, Gap: `150px`, Spawn interval: `90 frames`
- **Hard**: Speed: `6`, Gap: `125px`, Spawn interval: `70 frames`
- Keys: <kbd>1</kbd> for Easy, <kbd>2</kbd> for Medium, <kbd>3</kbd> for Hard, <kbd>Space</kbd> / <kbd>R</kbd> for Quick Replay, <kbd>Q</kbd> to quit.

Task 4: Sound Feedback Without External Download Failures

- Audio files (`flap.wav`, `score.wav`, `hit.wav`) were generated directly using Python's standard library `wave` and `math` modules.
- Wrapped inside [SoundManager](file:///C:/SE_LAB4/flappybird-updated/game/sound_manager.py) with `try/except` fallbacks so the game runs safely even on devices without audio hardware.

---

3. Updated Code Details

The updated code is already deployed and functional inside `C:\SE_LAB4\flappybird-updated\`. Here are the core modules:

A. Circle-to-AABB Collision & Bird: [`game/bird.py`](file:///C:/SE_LAB4/flappybird-updated/game/bird.py)

```
import pygame


class Bird:


    def __init__(self, x, y, radius=15):


        self.initial_x = x
```

```
        self.initial_y = y

        self.x = x

        self.y = y

        self.radius = radius

        self.velocity = 0

        self.gravity = 0.5

        self.flap_strength = -8

        self.max_fall_speed = 12



    def flap(self):

        self.velocity = self.flap_strength



    def update(self):

        self.velocity += self.gravity

        if self.velocity > self.max_fall_speed:

            self.velocity = self.max_fall_speed
```

```
        self.y += self.velocity



    def reset(self):

        self.x = self.initial_x

        self.y = self.initial_y

        self.velocity = 0



    def collides_with_rect(self, rect):

        """Task 1 Fix: Precise Circle-to-AABB collision detection."""

        closest_x = max(rect.left, min(self.x, rect.right))

        closest_y = max(rect.top, min(self.y, rect.bottom))

        dx = self.x - closest_x

        dy = self.y - closest_y

        return (dx  dx + dy  dy) <= (self.radius * self.radius)



    def render(self, screen):

        bx, by = int(self.x), int(self.y)
```

```
        # Bird body, wing, eye and beak rendering

        pygame.draw.circle(screen, (180, 140, 0), (bx, by), self.radius + 1)

        pygame.draw.circle(screen, (255, 215, 0), (bx, by), self.radius)

        wing_y_offset = -2 if self.velocity < 0 else 2

        pygame.draw.ellipse(screen, (240, 190, 0), (bx - 10, by + wing_y_offset, 12,

        pygame.draw.circle(screen, (255, 255, 255), (bx + 5, by - 5), 4)

        pygame.draw.circle(screen, (20, 20, 20), (bx + 6, by - 5), 2)

        pygame.draw.polygon(screen, (255, 120, 0), [(bx + 10, by - 2), (bx + 18, by
```

B. Sound Manager: [`game/sound_manager.py`](file:///C:/SE_LAB4/flappybird-updated/game/sound_manager.py)

```
import os
import pygame



class SoundManager:

    """Task 4: Sound effect manager with safe fallbacks."""
```

```
    def __init__(self):

        self.enabled = False

        self.sounds = {}

        try:

            if not pygame.mixer.get_init():

                pygame.mixer.init(frequency=44100, size=-16, channels=1, buffer=512)

            self.enabled = True

        except Exception:

            return



        sound_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "a

        for name in ["flap.wav", "score.wav", "hit.wav"]:

            path = os.path.join(sound_dir, name)

            if os.path.exists(path):

                self.sounds[name] = pygame.mixer.Sound(path)
```

```
    def play_flap(self):

        if self.enabled and "flap.wav" in self.sounds:

            self.sounds["flap.wav"].play()



    def play_score(self):

        if self.enabled and "score.wav" in self.sounds:

            self.sounds["score.wav"].play()



    def play_die(self):

        if self.enabled and "hit.wav" in self.sounds:

            self.sounds["hit.wav"].play()
```

C. Game Engine: [`game/game_engine.py`](file:///C:/SE_LAB4/flappybird-updated/game/game_engine.py)

Key features:

- **Task 1**: Circle collision with both top/bottom pipes, ceiling, and ground.
- **Task 2**: Game Over modal card with scores and instructions.

- **Task 3**: Dynamic difficulties (`Easy`, `Medium`, `Hard`) and in-place replay.
- **Task 4**: Sound feedback on flap, score, and death.

---

4. How to Run & Record the SECOND Video (Fixed & Working)

Running the Updated Game

Run in PowerShell:

```
cd C:\SE_LAB4\flappybird-updated
python main.py
```

What to Show in Your 10-Second Video:

1. **Audio Feedback (Task 4)**: Turn up sound while recording to demonstrate the jump chirp on flap and chime when clearing a pipe.
2. **Accurate Hit Detection (Task 1)**: As soon as any point of the bird's circular perimeter touches a pipe or the ground, death triggers with an impact sound.
3. **Interactive Game Over Screen (Task 2)**: The screen cleanly dims with a formatted card displaying **Final Score** and **Best Score**.
4. **Instant Replay & Difficulty Selection (Task 3)**:
5. Press <kbd>1</kbd> to immediately restart in **Easy** mode (slower pipes, wider 185px gap).
6. Or press <kbd>3</kbd> for **Hard** mode (faster pipes, narrower 125px gap).
7. Or press <kbd>Space</kbd> / <kbd>R</kbd> for quick replay at the current setting.

USER (STUDENT) 2026-10-15 06:56:09

can you change the updated version in a way like

1. once you run the command the game should open and then you should press start inorder to start the game
2. there should be an option to maximize the screen

AI ASSISTANT (PAIR-PROGRAMMING PARTNER) 2026-10-15 06:58:37

The updated game in `C:\SE_LAB4\flappybird-updated` has been enhanced with both features:

What Was Changed:

1. Interactive Start Screen (`game_engine.py`)

- **Idle State on Launch**: When you run `python main.py`, the game enters a dedicated **Start Screen** rather than immediately dropping into gameplay.
- **Animated Hovering Bird**: The bird hovers in an idle sine-wave bobbing animation while waiting for player input.
- **[START GAME] Button & Key Triggers**:
- Click the prominent green **START GAME** button with your mouse, OR
- Press <kbd>Space</kbd> or <kbd>Enter</kbd> to initiate the game.
- **Pre-Game Difficulty Selection**: You can choose **Easy**, **Medium**, or **Hard** before starting by either clicking the buttons on-screen or pressing <kbd>1</kbd>, <kbd>2</kbd>, or <kbd>3</kbd>.

2. Screen Maximize & Resizing Support (`main.py`)

- **Window Maximize Enabled**: The Pygame window is configured with `pygame.RESIZABLE`. The native Windows **Maximize button** on the top-right of the title bar is fully enabled.
- **Aspect-Ratio Preserved Scaling**: When maximized (or resized to any window size), the game dynamically scales via `pygame.transform.smoothscale` with clean letterboxing borders so the 500×700 retro aspect ratio never stretches or distorts.
- **Virtual Mouse Mapping**: When clicked on a maximized screen, mouse coordinates are automatically translated from window pixels back to the virtual canvas, allowing mouse clicks on **START GAME**, difficulty buttons, and flap clicks to hit the target.

- **Keyboard Shortcut**: You can also press <kbd>F11</kbd> or <kbd>Alt</kbd> + <kbd>Enter</kbd> at any time to toggle fullscreen/maximize mode.

How to Run

Open PowerShell or Command Prompt:

```
cd C:\SE_LAB4\flappybird-updated
python main.py
```

Controls Summary

| Action | Controls |

| :--- | :--- |

| **Start Game** | Click **START GAME** or press <kbd>Space</kbd> / <kbd>Enter</kbd> |

| **Difficulty Select** | Click buttons or press <kbd>1</kbd> (Easy), <kbd>2</kbd> (Medium), <kbd>3</kbd> (Hard) |

| **Flap** | <kbd>Space</kbd> or <kbd>Left Mouse Click</kbd> |

| **Maximize / Restore** | Window **Maximize Button** or <kbd>F11</kbd> |

| **Replay (Game Over)** | <kbd>1</kbd>, <kbd>2</kbd>, <kbd>3</kbd> or <kbd>Space</kbd> / <kbd>R</kbd> |

| **Quit** | <kbd>Q</kbd> or <kbd>Esc</kbd> |

Updated Files:

- [C:\SE_LAB4\flappybird-updated\main.py](file:///C:/SE_LAB4/flappybird-updated/main.py): Added `pygame.RESIZABLE`, aspect-ratio scaling, and virtual coordinate translation.
- [C:\SE_LAB4\flappybird-updated\game\game_engine.py](file:///C:/SE_LAB4/flappybird-updated/game/game_engine.py): Added `STATE_START` with start button, hovering animation, and click-to-start logic.
- [C:\SE_LAB4\flappybird-updated\README.md](file:///C:/SE_LAB4/flappybird-updated/README.md): Updated documentation.

USER (STUDENT) 2026-10-21 06:27:13

I NEED YOU TO DO THIS AGAIN

BUT FOR A DIFFERENT PERSIN

I HAVE DOWNLOADED THE ENTIRE REPO INTO THIS LOCATION

C:\LAB4_105

IT IS THERE IN THIS FILE AND THIS IS THE SRN:PES1UG24AM105

PLEASE COMEPLTE THIS TOO IN THE BEST WAY POSSIBLE

AI ASSISTANT (PAIR-PROGRAMMING PARTNER) 2026-10-21 06:33:11

The assigned repository for **SRN:** `PES1UG24AM105` is `39_endless-runner-main` (Real-Time Endless Runner Game).

Everything has been implemented, verified, and organized inside `C:\LAB4_105`.

Folder Setup at `C:\LAB4_105`

```
C:\LAB4_105\
├── endless-runner-original\           <-- [STARTER CODE with bugs to record Video 1]
│   ├── main.py
│   ├── README.md
│   ├── requirements.txt
│   └── game\
│       ├── game_engine.py
│       ├── obstacle.py
│       └── player.py
│
```

```
├── endless-runner-updated\            <-- [COMPLETED VERSION to record Video 2]
│   ├── main.py
│   ├── README.md
│   ├── requirements.txt
│   ├── assets\
│   │   └── sounds\
│   │       ├── jump.wav               <-- 16-bit PCM jump audio
│   │       ├── score.wav              <-- 16-bit PCM score chime
│   │       ├── hit.wav                <-- 16-bit PCM crash thud
│   │       └── generate_sounds.py
│   └── game\
│       ├── __init__.py
│       ├── game_engine.py             <-- Tasks 2 & 3: Game Over overlay & Replay/Dif
│       ├── obstacle.py                <-- Task 1: Continuous Swept Collision Detectio
│       ├── player.py                  <-- Player physics & running animation
```

```
|       └── sound_manager.py            <-- Task 4: Audio management

|

└── 39_endless-runner-main\             <-- Synced with the updated code (ready to pus
```

---

1. How to Run & Record the FIRST Video (Broken Behavior)

Command to Run the Original Code

Open PowerShell or Command Prompt:

```
cd C:\LAB4_105\endless-runner-original
python main.py
```

What to Show in Your 10-Second Video:

1. **No Start Screen or Audio**: The game drops straight into play immediately upon running with total silence (no jump, score, or death sound).
2. **Tunneling / High-Speed Collision Bug (Task 1)**:
3. The speed accelerates infinitely with no ceiling (`self.speed_increase_per_frame = 0.003`).
4. When speed is high, an obstacle covers more horizontal distance in a single frame than the player's 30px width. The obstacle can jump straight past the player without registering a collision.
5. **No Game Over Screen (Task 2 & Task 3)**:
6. When you hit an obstacle, the Pygame window abruptly freezes in place with no UI, no retry option, and only logs `Game over! Final score: X` to the terminal console.

> **Recording Tip**: Press <kbd>Win</kbd> + <kbd>Alt</kbd> + <kbd>R</kbd> (Windows Game Bar) or <kbd>Win</kbd> + <kbd>Shift</kbd> + <kbd>S</kbd> (Snipping Tool Screen Recorder) to record the 10-second clip.

---

2. All Tasks Completed & Architectural Changes

Task 1: Continuous Swept Collision Detection (CCD) & Speed Bounds

- **Problem**: Instantaneous box collision (`colliderect`) checks only the discrete position after the frame step.
- **Fix**: Implemented `swept_rect()` in [`game/obstacle.py`](file:///C:/LAB4_105/endless-runner-updated/game/obstacle.py#L22-L43) tracking the obstacle's horizontal displacement interval `[min(x, prev_x), max(x + w, prev_x + w)]`. If the swept path intersects the player while vertical clearance overlaps, a hit is registered with zero tunneling.
- Speeds are capped by difficulty level (`11.0` for Easy, `14.0` for Medium, `16.5` for Hard) with tuned progression.

Task 2: In-Game Game Over Screen with Student SRN

- Instead of abruptly freezing and logging only to the console, an in-game modal card renders over a darkened overlay showing:
- **GAME OVER** title
- **Final Score** & persistent **Best Score**
- Candidate identification: **`SRN: PES1UG24AM105`**
- Difficulty & replay prompts.

Task 3: Replay & Difficulty System

- In-place clean reset without restarting Python:
- **[1] Easy**: Speed `5.0`, spawn interval `85 frames`, max speed `11.0`

- **[2] Medium**: Speed `7.0`, spawn interval `65 frames`, max speed `14.0`
- **[3] Hard**: Speed `9.0`, spawn interval `50 frames`, max speed `16.5`
- **[SPACE / R]**: Quick replay at current difficulty
- **[Q / ESC]**: Exit cleanly

Task 4: Retro Sound Effects Feedback

- Generated standalone 16-bit PCM WAV audio files using Python's standard library `wave` and `math`:
- Jump sweep sound on jumping (<kbd>Space</kbd>/<kbd>W</kbd>/<kbd>Up</kbd>)
- Two-tone chime on scoring
- Impact thud on obstacle collision
- Managed by [`game/sound_manager.py`](file:///C:/LAB4_105/endless-runner-updated/game/sound_manager.py) with safe fallbacks.

Additional Features Added:

1. **Interactive Start Screen**:
2. Launches into a Title Screen: **"ENDLESS RUNNER"** with candidate banner `SRN: PES1UG24AM105`.
3. Prominent `[ START GAME ]` button and difficulty selector buttons (clickable with mouse or activated via keyboard).
4. **Window Maximize & Resizing Support**:
5. Uses `pygame.RESIZABLE` with the Windows **Maximize button** active on the title bar.
6. Smooth scaling (`pygame.transform.smoothscale`) preserving the 800×400 retro aspect ratio with clean letterboxing.
7. Auto-translates mouse coordinates for button clicking in maximized/fullscreen mode (<kbd>F11</kbd> toggle).

---

3. How to Run & Record the SECOND Video (Fixed & Working)

Command to Run the Completed Code

In PowerShell or Command Prompt:

```
cd C:\LAB4_105\endless-runner-updated

python main.py
```

What to Show in Your 10-Second Video:

1. **Start Screen**: Shows Title, `SRN: PES1UG24AM105`, and the **START GAME** button.
2. **Window Maximize**: Click the window **Maximize button** on the top-right of the title bar (or press <kbd>F11</kbd>) to show full-screen letterboxed scaling.
3. **Sound Effects**: Audio on jumping, passing obstacles, and impact.
4. **Collision Accuracy**: Continuous collision detection triggers reliably upon contact.
5. **Game Over Screen & Replay**: Shows Final Score, High Score, Candidate SRN, and pressing <kbd>1</kbd>, <kbd>2</kbd>, or <kbd>3</kbd> instantly restarts with different speeds.

---

4. Key Implementation Files

- [`C:\LAB4_105\endless-runner-updated\main.py`](file:///C:/LAB4_105/endless-runner-updated/main.py): Window maximize scaling, virtual coordinate mapping, and main game loop.
- [`C:\LAB4_105\endless-runner-updated\game\obstacle.py`](file:///C:/LAB4_105/endless-runner-updated/game/obstacle.py): Continuous swept collision detection and obstacle rendering.
- [`C:\LAB4_105\endless-runner-updated\game\game_engine.py`](file:///C:/LAB4_105/endless-runner-updated/game/game_engine.py): State machine (`START`, `PLAYING`, `GAME_OVER`), difficulty configurations, and SRN `PES1UG24AM105` display.

- [C:\LAB4_105\endless-runner-updated\game\sound_manager.py](file:///C:/LAB4_105/endless-runner-updated/game/sound_manager.py): Sound manager for jump, score, and hit audio.

USER (STUDENT) 2026-10-22 07:58:49

how to run these same files on a mac boo

AI ASSISTANT (PAIR-PROGRAMMING PARTNER) 2026-10-22 07:58:49

Here is the step-by-step guide to running these exact game files on a **MacBook (macOS)**:

Step 1: Copy the Project Folder to Your Mac

Transfer your project folder (e.g. `endless-runner-updated` or `flappybird-updated`) to your Mac using AirDrop, a USB drive, Google Drive, or GitHub.

- A good location is your **Desktop** or **Downloads** folder (e.g. `~/Desktop/endless-runner-updated`).

Step 2: Open Terminal & Check Python

1. Press <kbd>Cmd</kbd> + <kbd>Space</kbd> to open Spotlight, type **Terminal**, and press <kbd>Return</kbd>.
2. Check if Python 3 is installed:

   ```
   python3 --version
   ```

   (If Python is not installed, macOS will automatically prompt you to install Apple Command Line Tools, or you can install it from [python.org](https://www.python.org/downloads/)).

   Step 3: Install Pygame on macOS

On modern macOS versions, the cleanest and recommended way to install packages is inside a virtual environment:

1. Navigate to your project folder in Terminal:

```
cd ~/Desktop/endless-runner-updated
```

(Replace with the actual path where you placed the folder).

1. Create and activate a virtual environment:

```
python3 -m venv venv


   source venv/bin/activate
```

1. Install Pygame:

```
pip install pygame
```

(Or if you are not using a virtual environment: `pip3 install pygame`).

Step 4: Run the Game

While inside the project directory in Terminal:

```
python3 main.py
```

- The Pygame window will open immediately on your Mac.

Step 5: How to Record the 10-Second Videos on a Mac

macOS has a built-in screen recorder with zero setup needed:

1. Press **<kbd>Cmd</kbd> + <kbd>Shift</kbd> + <kbd>5</kbd>**.
2. Select **"Record Selected Portion"** and drag the box over your game window (or choose "Record Entire Screen").
3. Click **Record**.
4. Play the game for 10 seconds.
5. Click the small **Stop** button in the top macOS menu bar (next to the WiFi and Clock icons).
6. The video file will automatically save directly to your **Desktop** as a `.mov` file.

💡 **Helpful Mac-Specific Tips**

- **Maximize Screen**: Click the **green button** in the top-left corner of the window to expand/maximize it.
- **Fullscreen Shortcut**: To toggle fullscreen with <kbd>F11</kbd> on a Mac keyboard, press **<kbd>Fn</kbd> + <kbd>F11</kbd>** (since the top row keys default to volume/brightness controls unless <kbd>Fn</kbd> is held).
- **Quitting**: You can quit at any time using the in-game key <kbd>Q</kbd> / <kbd>Esc</kbd>, or by pressing <kbd>Cmd</kbd> + <kbd>Q</kbd>.

USER (STUDENT)

2026-10-22 09:00:29

c. Chat history exported as a doc/pdf

this is one of the deliverables that i found

how to do it

AI ASSISTANT (PAIR-PROGRAMMING PARTNER) 2026-10-22 09:02:37

The complete chat history has been exported as both a **ready-to-submit PDF** and a **formatted document** inside your lab folder:

📂 **Your Exported Chat History Files:**

1. **PDF File (Ready for Submission)**:
2. **Location**: [`C:\LAB4_105\Lab4_Chat_History_PES1UG24AM105.pdf`] (file:///C:/LAB4_105/Lab4_Chat_History_PES1UG24AM105.pdf)
3. Formatted with a submission header banner (Student SRN: `PES1UG24AM105`, assigned repo: `39_endless-runner-main`, date, timestamps, styled user prompts, assistant replies, and syntax-highlighted code blocks).

    1. **HTML / Word Document File**:
    2. **Location**: [`C:\LAB4_105\Lab4_Chat_History_PES1UG24AM105.html`] (file:///C:/LAB4_105/Lab4_Chat_History_PES1UG24AM105.html)

    *Can be opened directly in **Microsoft Word** or **Google Docs** (Right-click $\rightarrow$ Open with $\rightarrow$ Word*) and saved as* `.docx`.

    (A backup copy has also been placed in `C:\SE_LAB4\Lab4_Chat_History.pdf`*.)*

    How to Manually Export Any Chat to PDF / Doc (General Method):

If you ever need to export chats yourself from any browser or AI tool in the future, follow these two standard methods:

Method 1: Browser "Print to PDF" (Fastest & Standard)

1. In your browser tab with the chat open, press:
2. **Windows**: <kbd>Ctrl</kbd> + <kbd>P</kbd>
3. **Mac**: <kbd>Cmd</kbd> + <kbd>P</kbd>
4. In the print dialog, change the **Destination** from your physical printer to **"Save as PDF"**.
5. Under *More settings*, check **"Background graphics"** (so code boxes and colored text render properly).
6. Click **Save** and choose your Lab folder.

Method 2: Convert to Word (.doc / .docx)

1. Open the exported `.html` file (or select and copy all text from the chat).
2. Right-click [`Lab4_Chat_History_PES1UG24AM105.html`](file:///C:/LAB4_105/Lab4_Chat_History_PES1UG24AM105.html) $\rightarrow$ **Open with** $\rightarrow$ **Microsoft Word**.
3. Go to **File** $\rightarrow$ **Save As** $\rightarrow$ choose **Word Document (*.docx)**.

✅ **Your Lab-4 Submission Checklist Status**

All required deliverables are completed in `C:\LAB4_105`:

Deliverable	File in `C:\LAB4_105`	Status

| **a. Videos (Before & After)** | `2026-10-22 14-27-13.mp4`
 `2026-10-22 14-28-04.mp4` | Recorded |

| **b. Updated Code** | `endless-runner-updated/`
 `39_endless-runner-main/` | Completed |

| **c. Chat History (.pdf / .doc)** | `Lab4_Chat_History_PES1UG24AM105.pdf`
 `Lab4_Chat_History_PES1UG24AM105.html` | Exported |

USER (STUDENT) 2026-10-22 09:04:08

i need it for the 106 also

the flappy bird one